Payment Auto-Approver App: Full Build Spec

For: Antigravity / Gemini. Platform: Android only (Flutter). Local DB: Hive.

0. Instruction to the AI builder (read first)

Build a Flutter Android app that runs on a dedicated phone holding a bKash Agent SIM and a Nagad Agent SIM. The app reads incoming Cash Out SMS from bKash and Nagad, saves them in Hive, fetches a Pending Payment list from my server API, matches the SMS Transaction ID with the pending list Transaction ID, and calls my Approve API for every match. It must keep working in the background and must also catch up on everything missed when the app is opened.

Authentication: my server APIs need an admin token. The app MUST start with a Login screen that calls my Login API, stores the token securely, and attaches it to every Pending and Approve API call. Details: Section 4A (flow), Section 5 Screen 0 (UI), Section 6.1 (Login API).

Rules for the builder:

- Build phase by phase (Section 14). Make each phase run before starting the next.
- Keep all server API details in ONE file (`api_config.dart` + `payment_api.dart`) because the real JSON shapes are filled in later (Section 6).
- Never approve the same transaction twice (idempotency, Section 8).
- Android only. iOS cannot read SMS.

1. Goal and Flow

1. Customer pays an agent via Cash Out. The agent phone receives an SMS from bKash or Nagad.
2. App parses the SMS and saves it to Hive with status `unmatched` .
3. Every 5 minutes (and on a manual Sync tap) the app calls the Pending Payment API.
4. For every pending payment, the app looks for an SMS record with the same Transaction ID.
5. If it matches, the app calls the Approve API and marks the record `approved` .
6. If it does not match, nothing happens and the record stays `unmatched` until a later sync.

Precondition for steps 3 to 5: the admin must be logged in with a valid token. Without a valid session the app still reads and saves SMS, but it calls no server API (see Section 4A).

2. Tech Stack and Packages

Purpose	Package (suggested)
State management	flutter_riverpod (or provider)
Local DB	hive, hive_flutter, hive_generator, build_runner
Read SMS inbox and receive SMS	another_telephony (or telephony). Fallback: flutter_sms_inbox + a native Kotlin BroadcastReceiver
Background execution	flutter_background_service (foreground service + 5 min timer)
Notification for foreground service	flutter_local_notifications
HTTP	dio
Permissions	permission_handler
Battery optimization exemption	disable_battery_optimization (or permission_handler ignoreBatteryOptimizations)
Secure token storage	flutter_secure_storage
Date formatting	intl
Connectivity check	connectivity_plus

Android: minSdk 23+, target the latest stable SDK.

3. SMS Formats (REAL samples, parse these exactly)

3.1 Nagad Cash Out SMS

```
Cash Out Received.
Amount: Tk 8900.00
Customer: 0182***5031
TxnID: 766T57YC
Comm: Tk 36.49
Balance: Tk 56331.66
22/10/2026 16:50
```

Fields to extract: amount = 8900.00, customer = 0182***5031, txnId = 766T57YC, commission = 36.49, balance = 56331.66, smsDateTime = 22/10/2026 16:50 (format dd/MM/yyyy HH:mm).

Suggested regexes (case-insensitive):

- Detect: text contains `Cash Out Received`
- Amount: `Amount:\s*Tk\s*([\d,]+(?:\.\d+)?)`
- Customer: `Customer:\s*([\d\*Xx]+)`
- TxnID: `TxnID:\s*([A-Z0-9]+)`
- Commission: `Comm:\s*Tk\s*([\d,]+(?:\.\d+)?)`
- Balance: `Balance:\s*Tk\s*([\d,]+(?:\.\d+)?)`
- DateTime: `(\d{2}/\d{2}/\d{4}\s+\d{2}:\d{2})`

3.2 bKash Cash Out SMS

```
Cash Out Tk 1,000.00 from 0133XXXX032 successful. Fee Tk 0.00. Balance Tk
223,849.23. TrxID DJP4HZUWF0 at 25/10/2026 07:46
```

Fields to extract: amount = 1000.00 (remove the comma), customer = 0133XXXX032, fee = 0.00, balance = 223849.23, txnId = DJP4HZUWF0, smsDateTime = 25/10/2026 07:46.

Suggested regexes:

- Detect: text matches `^Cash Out Tk` and contains `TrxID`
- Amount: `Cash Out Tk\s*([\d,]+(?:\.\d+)?)`
- Customer: `from\s*([\d\*Xx]+)`
- Fee: `Fee Tk\s*([\d,]+(?:\.\d+)?)`

- Balance: `Balance Tk\s*([\d,]+(?:\.\d+)?)`
- TxnID: `TrxID\s*([A-Z0-9]+)`
- DateTime: `at\s*(\d{2}/\d{2}/\d{4}\s+\d{2}:\d{2})`

3.3 Parser rules

- Build an abstract `SmsParser` with `bool canParse(sender, body)` and `ParsedSms? parse(sender, body, receivedAt)`. Implement `NagadCashOutParser` and `BkashCashOutParser`. Adding a new SMS type later (Cash In, Payment, etc.) must only need a new parser class.
- Strip commas from numbers before converting to double.
- Normalize txnId: trim and uppercase. Matching always uses the normalized value.
- Sender IDs are probably `bKash` and `NAGAD` but must be configurable in Settings (default list: bKash, NAGAD, 16167, 16216). Verify on the real phone and let the user edit the list.
- If an SMS from these senders does not match any parser, ignore it (do not crash, only write a debug log).
- Store BOTH the raw SMS body and the parsed fields.
- Dedup key: `provider + txnId`. Same SMS must never create two records, even if both the live receiver and the inbox scan see it.
- Timezone: Bangladesh (UTC+6). Store the SMS date as parsed from the text AND the device receive timestamp.

4. Hive Data Models

Register a TypeAdapter for each. Use the Hive key `provider_txnId` for SmsRecord (this gives free dedup).

4.1 SmsRecord (box: `sms_records`)

Field	Type	Note
id	String	provider_txnId (box key)
provider	String	`bkash` or `nagad`
sender	String	SMS sender id

Field	Type	Note
rawBody	String	full SMS text
txnId	String	normalized
amount	double	
customer	String	masked number
fee	double?	bKash
commission	double?	Nagad
balance	double?	
smsDateTime	DateTime	from SMS text
receivedAt	DateTime	device time
status	String	see 4.2
matchedPaymentId	String?	server payment id
approvedAt	DateTime?	
failReason	String?	last approve error
attempts	int	approve attempts
approvedBy	String?	`auto` or `manual`

4.2 Status values

- `unmatched` : saved, no pending payment found yet
- `matched` : found in pending list, approve not done yet
- `approved` : approve API succeeded
- `failed` : approve API failed (will retry, see Section 8)
- `amount_mismatch` : txnId matched but amount differs; needs manual review (only when strict amount check is ON)

4.3 PendingPayment (box: `pending_payments`)

A cache of the last fetched server list. Replace the whole box content after each successful fetch. Fields: paymentId, txnId (normalized), amount, customerName/phone (if the API gives it), method, createdAt, rawJson, lastFetchedAt.

4.4 AppState / Settings (box: `app_state`, single object)

- lastSmsReadAt (DateTime), the last time the inbox was read
- lastSmsTimestampProcessed (int, millis of the newest SMS processed; used for incremental inbox scan)
- lastSyncAt, lastSyncStatus (`success` or `failed`), lastSyncMessage
- settings: pendingApiUrl, approveApiUrl, baseUrl, loginPath (the auth token itself is kept in secure storage, see Section 4A), syncIntervalMinutes (default 5), strictAmountCheck (default true), autoApprove (default true), senderIds list, initialScanDays (default 7)

4.5 SyncLog (box: `sync_logs`)

One row per sync run: id, startedAt, finishedAt, trigger (`timer`, `manual`, `app_open`, `sms_event`), smsScanned, newSmsSaved, pendingFetched, matchedCount, approvedCount, failedCount, error. Keep the last 200 rows and delete older ones.

4A. Authentication (Admin Login)

The Pending and Approve APIs need an admin token, so the app starts with a Login screen. No sync, matching or approve call may run without a valid session.

Login flow

1. The admin enters the credentials the Login API expects (email, username or phone, plus password) on Screen 0.
2. The app calls the Login API (Section 6.1). On success it receives an access token and, depending on the server, a refresh token, an expiry time and the admin profile.
3. The app saves these in flutter_secure_storage ONLY (never Hive, never SharedPreferences, never logs): accessToken, refreshToken (optional), tokenExpiresAt (optional), adminId, adminName, adminRole, baseUrl.
4. The session state is kept in a small `AuthState` object with values `loggedOut`, `active`, `expired`.

5. ALL server calls go through ONE dio client whose interceptor adds the header `Authorization: Bearer <accessToken>`. Header name and scheme are set in `api_config.dart` in case my server uses something different.
6. App start (Splash screen acts as the auth gate): no token means Login screen; token with a known expiry in the past means try refresh, else Login; otherwise continue to Permission Setup (if permissions are missing) or Home.
7. If the login response contains a role and it is not admin, show This account is not an admin and do not save the token.

Token expiry and 401 handling

- On 401 or 403 from the Pending or Approve API: if a Refresh API exists, call it once, save the new tokens and retry the failed call once.
- If there is no Refresh API, or the refresh fails: set session = `expired`. SMS reading and saving to Hive CONTINUE, but auto-sync and auto-approve STOP.
- Show a red banner on Home (Session expired. Login again) and send ONE local notification (do not repeat it every 5 minutes).
- When the app opens in the `expired` state it goes to the Login screen. After a successful login, run a sync immediately so the queued SMS get matched and approved.
- Optional setting **Keep me signed in (auto re-login)**, default OFF: stores the password in secure storage and silently logs in again when the token expires. Show a warning about the security risk. Reason it exists: this phone runs unattended, so a short-lived token would stop approvals until someone logs in. Better option: ask the server for a long-lived admin token or a refresh token.

Logout

- Settings has a Logout button with a confirm dialog.
- It calls the Logout API if one exists (optional), deletes all tokens from secure storage, and sets session = `loggedOut`.
- Ask Keep local SMS data? (default: keep). The background service keeps capturing SMS but skips all API calls while logged out.

Security rules

- HTTPS only. Reject `http://` base URLs in release builds.
- Never print the token or password in logs, crash reports or SyncLog rows.

- The password field is obscured with a show/hide toggle.
- After 5 wrong logins in a row, disable the Login button for 30 seconds.
- Optional: block screenshots on the Login screen (FLAG_SECURE).

5. Screens (8 total) and Navigation

Use a Bottom Navigation Bar with 5 tabs: Home, SMS, Pending, Approve, Settings. Plus a Splash/Login screen (Screen 0), a first-run Permission Setup screen and a Sync Logs screen opened from Settings. App start order: Splash (auth gate), then Login if there is no valid token, then Permission Setup if permissions are missing, then Home. The 5 tabs are visible only when the session is active.

Screen 0: Splash and Login (first screen of the app)

- Splash: logo, then the auth gate from Section 4A (checks the saved token and decides the next screen).
- Login screen fields: Email / Username / Phone (whichever the Login API needs), Password (show/hide toggle), and a collapsed **Advanced** section with the Server Base URL (prefilled from `api_config.dart`).
- **Login** button with a loading state. Show the server error message (for example Invalid credentials) in a red text under the form. Show a clear message for no internet and for timeout.
- A **Remember username** checkbox (stores only the username, never the password).
- After success: go to Permission Setup (first run) or Home, and trigger a first sync.
- The same screen is shown whenever the session is `expired` or `loggedOut`. It must open on top of everything and block the main tabs until login succeeds.

Screen 1: Permission Setup (first run, and re-openable from Settings)

Steps with a status tick for each:

1. SMS permission (READ_SMS, RECEIVE_SMS)
2. Notification permission (Android 13+)
3. Disable battery optimization for this app
4. Autostart guidance for Xiaomi, Oppo, Vivo, Samsung, Realme, Huawei (a text instruction card with an Open Settings button)

5. Start the background service The app can only continue to Home when SMS permission and the background service are OK.

Screen 2: Home / Dashboard

- Top card: **Last SMS read at** (date and time) plus the text, for example, 2 minutes ago.
- Card: **Last sync** time and result (success or failed with message).
- Background service status chip (Running, Stopped) with a toggle or restart button.
- Counters for today and total: SMS read, total amount, Approved, Unmatched, Failed.
- Big **Sync Now** button (shows a progress indicator, disabled while syncing).
- Small list of the latest 5 SMS records with a status chip.
- Pull to refresh runs the same as Sync Now.

Screen 3: SMS Records

- Full list of all saved SmsRecords, newest first.
- Each row: provider logo/color (bKash pink, Nagad orange), amount, txnId, masked customer, SMS time, status chip.
- Filters: provider (All, bKash, Nagad), status (All, Unmatched, Matched, Approved, Failed, Amount mismatch), date range, and a search box (txnId or amount).
- Top summary: count and total amount for the current filter.
- Tap a row for a bottom sheet with all fields including the raw SMS, matched payment id, approve attempts, failReason. Include a Copy txnId button.
- Button **Read SMS from phone**: manually triggers an inbox scan (Section 9) with a date range option (Today, 3 days, 7 days, custom). It saves only new records.
- This list is the Hive data directly; it is the single source of truth.

Screen 4: Pending Payments (view only)

- Shows the PendingPayment list from the server (cached in Hive, with a Refresh button that re-calls the API).
- Each row: txnId, amount, customer, created time, and a **match badge**: Matched with SMS (green), No SMS yet (grey), Amount mismatch (orange).
- Top bar shows count, total amount, and the last fetched time.
- Search by txnId.

Screen 5: Manual Approve

- Lists pending payments that have a matching SMS record but are not approved yet (matched, failed, amount_mismatch). A toggle at the top, **Show all pending**, also shows the ones without SMS.
- Each row: txnId, amount (server vs SMS amount side by side), the SMS time, and an **Approve** button.
- Approve shows a confirm dialog with the details. On confirm, it calls the Approve API, then updates the status to `approved` with approvedBy = `manual`.
- Multi-select with **Approve Selected** for bulk approval, plus a confirm dialog.
- For pending payments with no SMS, the Approve button shows a strong warning dialog (**No SMS found. Approve anyway?**). This is optional and can be hidden behind a setting `allowForceApprove` (default OFF).
- Show the API error message inline if an approve fails, with a Retry button.

Screen 6: Settings

- API: Pending API URL, Approve API URL, Base URL, the logged-in admin name and role (read only), a Test Connection button (calls the Pending API with the saved token), and a Logout button (see Section 4A).
- Sync: interval in minutes (default 5, minimum 1), Auto-approve ON/OFF, Strict amount check ON/OFF.
- SMS: editable sender ID list, initial scan days.
- Background: Service status, Permission Setup shortcut, battery optimization shortcut.
- Data: Export SMS records as CSV, Clear synced old records (older than N days), Clear all data (with a confirm dialog).
- Open Sync Logs.

Screen 7: Sync Logs

List of SyncLog rows (time, trigger, counts, error). Tap to expand.

6. Server API Contract (PLACEHOLDER: I will fill the real shapes)

I have three APIs on my server: the Login API (Section 6.1, used first to get the admin token) and the two below, which both need that token:

1. **Pending Payment List API**: returns all payments waiting for approval.

2. **Approve Payment API**: approves one payment (or several).

Until I give the real format, implement against this ASSUMED shape and isolate the mapping in `payment_api.dart` so changing it later touches one file only:

```
GET {pendingApiUrl}
Header: Authorization: Bearer <token>
Response: { "data": [ { "id": "123", "transaction_id": "766T57YC", "amount":
8900, "customer": "01XXXXXXXXX", "method": "nagad", "created_at": "2026-10-
22T16:40:00Z" } ] }

POST {approveApiUrl}
Header: Authorization: Bearer <token>
Body: { "id": "123", "transaction_id": "766T57YC" }
Response: { "success": true, "message": "Approved" }
```

Rules:

- Request timeout 20 seconds. Retry on network errors (see Section 8).
- Treat the server response saying already approved as SUCCESS (idempotent) and mark the local record `approved`.
- Add an `ApiException` with a readable message for the UI.
- Never log the auth token.

6.1 Login API (PLACEHOLDER: needed before the two APIs above)

The endpoint, base URL and headers below are CONFIRMED from my project docs (ADMIN_API.md has the short list, API_GUIDE.md has the details). The request body and response fields are still NOT confirmed, so implement them as the ASSUMED shape in `auth_api.dart` only, so changing it later touches one file:

Confirmed details

- Local base URL: `http://localhost:8001/api/v1`
- Production base URL: `https://<your-api-host>/api/v1` (placeholder, I will give the real host; keep it editable)
- Login: `POST /auth/login`, no Authorization header needed, header `Content-Type: application/json`
- Defaults in `api_config.dart`: `loginPath = /auth/login`; `baseUrl` = the local URL in debug builds and the production URL in release builds; the user can override it in Login Advanced and in Settings.

Base URL notes for testing on a phone

- `localhost` on a phone means the phone itself, not my computer. On a real phone use my computer's LAN address on the same Wi-Fi, for example `http://192.168.x.x:8001/api/v1`. On the Android emulator use `http://10.0.2.2:8001/api/v1`.
- Android blocks plain `http://` by default. Allow it ONLY in the debug build (`android:usesCleartextTraffic="true"` in `android/app/src/debug/AndroidManifest.xml`). The release build must use the HTTPS production URL (matches the HTTPS-only rule in Section 4A).
- Make sure the server listens on `0.0.0.0:8001` (not only 127.0.0.1) and the computer firewall allows port 8001, otherwise the phone cannot reach it.

Assumed body and response (not confirmed yet)

```
POST {baseUrl}{loginPath}
Body: { "email": "admin@example.com", "password": "******" }
Response 200: { "success": true, "data": { "token": "<jwt>", "refresh_token":
"<optional>", "expires_in": 86400, "user": { "id": "1", "name": "Admin",
"role": "admin" } } }
Response 401: { "success": false, "message": "Invalid credentials" }
```

Rules:

- Credential field names (email, username or phone) and the response paths (`data.token`, `data.refresh_token`, `data.expires_in`, `data.user`) live only in `auth_api.dart`.
- Optional endpoints, implemented only if my server has them: Refresh API (`POST {refreshPath}` with the refresh token) and Logout API.
- If the server answers that an OTP or 2FA code is needed, show a clear message. OTP is not supported in version 1.
- Pending and Approve calls use the token saved by this login (Section 4A).

6.2 API Reference (pasted from my project docs, as provided)

হ্যাঁ—প্রজেক্টে তিনটিই আছে। নিচে local base URL দিয়ে endpoint-গুলো দিলাম। ADMIN_API.md-তে সংক্ষিপ্ত তালিকা, আর API_GUIDE.md-তে বিস্তারিত আছে।

Base URL

- Local: `http://localhost:8001/api/v1`

- Production: `https://<your-api-host>/api/v1` — production host docs-এ placeholder হিসেবে আছে।

1. Admin login — আছে

- `POST /auth/login`
- Auth: লাগে না
- Header: `Content-Type: application/json`

The pasted text ends here. The other two endpoints (Pending Payment list and Approve Payment) and the login request and response bodies are not provided yet. When I paste them, add them to this section in the same format and update Section 6 and 6.1 to match.

7. Core Matching Logic

```
for each pending in pendingList:
    sms = smsBox.findByTxnId(pending.txnId)   // normalized, case-insensitive
    if sms == null: continue                  // wait for a later sync
    if sms.status == approved: continue       // never approve twice
    if strictAmountCheck and sms.amount != pending.amount:
        sms.status = amount_mismatch; save; continue
    sms.status = matched; sms.matchedPaymentId = pending.id; save
    if autoApprove:
        approve(pending)
```

Notes:

- Compare amounts with a tolerance of 0.01 (double rounding).
- If a txnId appears twice in the pending list, approve once and log a warning.
- Match on provider too if the server gives a method field (optional).

8. Approve, Retry and Safety Rules

- Before each approve call: re-read the record from Hive. If the status is already `approved`, skip.
- On success: status = `approved`, approvedAt = now, approvedBy = `auto` or `manual`, attempts += 1.

- On failure: status = `failed`, failReason = message, attempts += 1. The next sync retries `failed` records automatically up to 5 attempts. After 5, they stay `failed` and appear only in Manual Approve with a red badge.
- A global lock (`isSyncing`) so two syncs never run at the same time (timer + manual + SMS event). If a sync is already running, a new request is queued or skipped.
- Network offline: skip the API calls, still save SMS locally, log `offline`, retry on the next tick.
- 401 or 403: follow the token expiry procedure in Section 4A (refresh once and retry once; if that fails, set the session to expired, stop auto-approve, show a Session expired banner on Home and send one notification).
- Approve sequentially (one by one), not all in parallel, to avoid server overload.

9. Reading SMS: three paths that all feed the same pipeline

All three paths call the same function `SmsIngestService.processSms(sender, body, timestamp)` which parses, dedups and saves.

1. **Live receiver (foreground and background):** register an SMS broadcast listener. When an SMS arrives from a configured sender, process it immediately and trigger a short-delay sync (debounce 10 seconds).
2. **Periodic timer (background service):** every 5 minutes run a full sync (Section 10).
3. **Inbox scan (catch up):** query the SMS inbox for messages from the configured senders with date greater than `lastSmsTimestampProcessed`. Run it:
   - every time the app is opened or resumes,
   - at the start of every sync,
   - when the user taps **Read SMS from phone**.
   - On the very first run, scan back `initialScanDays` (default 7). After processing, update `lastSmsReadAt = now` and `lastSmsTimestampProcessed = newest SMS timestamp`.

This is what covers the case where the app was not running in the background: when it opens, the inbox scan reads every Cash Out SMS received meanwhile, and the sync then matches and approves them.

10. Sync Procedure (one function used by timer, manual button, app open and SMS event)

```
sync(trigger):
  if isSyncing: return
  isSyncing = true; log = new SyncLog(trigger)
  1. inboxScan()                      // read new SMS, save new records
  2. if offline: finish with status offline
  3. pending = api.fetchPending()     // cache into pending_payments box
  4. runMatching(pending)             // Section 7
  5. retry failed records (attempts < 5)
  6. update AppState.lastSyncAt/lastSyncStatus; save SyncLog
  7. notify UI (stream/event)
  isSyncing = false
```

11. Running in Background (Android): what the builder MUST do

1. **Foreground service** (flutter_background_service, `isForegroundMode: true`, notification channel `sync_service`) with a persistent notification such as: Payment Sync running. Last sync: HH:mm. Update the text after each sync. A foreground service is what keeps Android from killing the app. Set `autoStart: true` and `autoStartOnBoot: true`.
2. **5 minute timer inside the service:** use `Timer.periodic(Duration(minutes: syncIntervalMinutes))`. Do NOT rely on WorkManager for the 5 minute loop, because its minimum interval is 15 minutes. WorkManager (15 min) may be added only as a backup watchdog that restarts the service if it died.
3. **Boot receiver:** restart the service after phone reboot (RECEIVE_BOOT_COMPLETED).
4. **Wake lock:** hold a partial wake lock only while a sync runs.
5. **Battery optimization:** request ignore-battery-optimization exemption in Permission Setup. Show OEM-specific autostart instructions (Xiaomi, Oppo, Vivo, Realme, Samsung, Huawei).
6. **SMS in background:** with the telephony package use the `onBackgroundMessage` top-level function (annotated `@pragma('vm:entry-point')`) so SMS received while the UI is closed are still processed.
7. **Isolate and Hive warning (IMPORTANT):** the background service and the SMS background handler run in separate isolates from the UI. Hive does not safely allow two isolates to write the same box. Required design:

- The background isolate is the ONLY writer to Hive. It calls `Hive.initFlutter()` and opens the boxes itself.
- The UI isolate sends commands (sync now, manual approve) to the service through `service.invoke('syncNow')` and listens to events from `service.on('update')`.
- When the UI receives an update event, it closes and re-opens the boxes (or calls the box `compact`/reopen approach) to see fresh data, then refreshes the screens.
- Where the SMS background handler (separate isolate) cannot reach the service, it must append raw SMS into a small queue box or SharedPreferences list; the service drains that queue into Hive on its next tick, and the inbox scan also covers any gaps. Dedup guarantees no duplicates.
- If this cross-isolate setup proves unstable, report it and suggest switching the DB to Isar or SQLite (drift) instead of fighting Hive.

8. **Play Store note:** READ_SMS and RECEIVE_SMS are restricted by Google Play. This app is for private use, so distribute as an APK (sideload). Do not target the Play Store.
9. Keep the screen-off, locked, and battery-saver states in mind: test with the screen off for 30+ minutes.

Auth in the background service: the service reads the token from flutter_secure_storage on every sync (not once at start), because the user may log in or out while it runs. Before any API call it checks the session state; if it is `loggedOut` or `expired` it only saves SMS and skips the API steps. A 401 in the background follows the procedure in Section 4A. Test that secure storage reads work inside the background isolate.

12. AndroidManifest permissions

- INTERNET, ACCESS_NETWORK_STATE
- READ_SMS, RECEIVE_SMS
- FOREGROUND_SERVICE, FOREGROUND_SERVICE_DATA_SYNC (Android 14+; declare the service with foregroundServiceType=dataSync)
- POST_NOTIFICATIONS
- RECEIVE_BOOT_COMPLETED
- WAKE_LOCK
- REQUEST_IGNORE_BATTERY_OPTIMIZATIONS

13. Suggested Folder Structure

```
lib/
  main.dart
  core/ (constants, theme, utils, api_config.dart)
  data/
    models/ (sms_record.dart, pending_payment.dart, app_state.dart,
sync_log.dart)
    hive/ (hive_service.dart, adapters)
    api/ (payment_api.dart, api_exception.dart)
    sms/ (sms_reader.dart, parsers/{sms_parser.dart, nagad_parser.dart,
bkash_parser.dart})
  services/
    background_service.dart
    sync_service.dart
    matching_service.dart
    approve_service.dart
    notification_service.dart
    permission_service.dart
  providers/ (riverpod providers)
  ui/
    permission_setup/
    home/
    sms_records/
    pending/
    manual_approve/
    settings/
    sync_logs/
    widgets/ (status_chip.dart, provider_badge.dart, amount_text.dart)
```

Add for authentication: `lib/data/auth/` (auth_api.dart, auth_repository.dart, token_storage.dart, auth_state.dart), `lib/core/http/dio_client.dart` (the single dio client with the auth interceptor), `lib/ui/splash/` and `lib/ui/login/`.

14. Build Phases (give to the AI one at a time)

1. **Project setup:** packages, manifest permissions, Hive init and models with adapters, theme, bottom navigation with empty screens.
2. **SMS parsing:** the two parsers plus unit tests using the exact samples in Section 3. Tests must pass.

3. **Permission Setup screen** and SMS inbox reader with incremental scan and dedup. SMS Records screen with filters and the **Read SMS from phone** button.
4. **API layer** with the assumed contract, Settings screen with Test Connection, Pending Payments screen.
5. **Matching and approve engine** with the sync function, retry rules and the SyncLog. Home screen with Sync Now and counters. Manual Approve screen.
6. **Background service:** foreground service, 5 min timer, boot start, live SMS receiver, single-writer Hive design (Section 11), notification updates.
7. **Hardening:** offline handling, 401 handling, battery guide, CSV export, old data cleanup, and a test plan run on a real phone (screen off, reboot, offline, duplicate SMS, app swiped away).

Phase 4b (authentication, do it right after Phase 4 and before Phase 5): build `token_storage.dart` with flutter_secure_storage, `auth_api.dart` with the assumed Login API, the single dio client with the auth interceptor and 401 handling, the Splash auth gate, the Login screen, and Logout in Settings. Switch the Pending and Approve calls to use this dio client. Test: wrong password, correct login, app restart keeps the session, forced 401 shows the Session expired banner, and re-login resumes sync.

15. Acceptance Checklist

- [ ] Both sample SMS parse to the exact values in Section 3.
- [ ] The same SMS processed twice creates only one record.
- [ ] With the app swiped away and the screen off, a new SMS is saved and the sync runs every 5 minutes.
- [ ] After phone reboot, the service restarts without opening the app.
- [ ] After the app was fully dead for hours, opening it reads all missed Cash Out SMS and Home shows the correct **last SMS read** time.
- [ ] A matching txnId triggers exactly one approve call; failed ones retry up to 5 times.
- [ ] Manual Approve works for single and bulk selection.
- [ ] Sync Now never runs two syncs at once.
- [ ] Strict amount mismatch goes to Amount mismatch and is not auto-approved.
- [ ] The auth token is stored securely and never logged.
- [ ] Login works; the token is stored only in secure storage and survives an app restart.
- [ ] Pending and Approve calls always carry the token; a missing token blocks them.

- [ ] On 401 the app tries refresh once; otherwise the session becomes expired, auto-approve stops, SMS saving continues, and one notification is shown.
- [ ] After re-login a sync runs immediately and the queued SMS are matched.
- [ ] Logout clears the tokens and stops all API calls.

16. Info I still need to provide to the builder

- Real Pending API: URL, method, auth type, and a sample JSON response.
- Real Approve API: URL, method, request body, success and already-approved response samples.
- Exact SMS sender IDs seen on the phone for bKash and Nagad.
- Whether the amount must always match exactly.
- Whether other SMS types (Cash In, Payment, Send Money) need to be handled later.

Login-related info I still need to provide:

- Login API: the credential field names in the request body (email, username or phone) and sample success and error responses. The URL (POST /auth/login), base URL and headers are already confirmed.
- Token details: header format (Bearer or other), token lifetime, and whether a Refresh API exists.
- Whether a Logout API exists, and whether the login response includes a role to check for admin.